

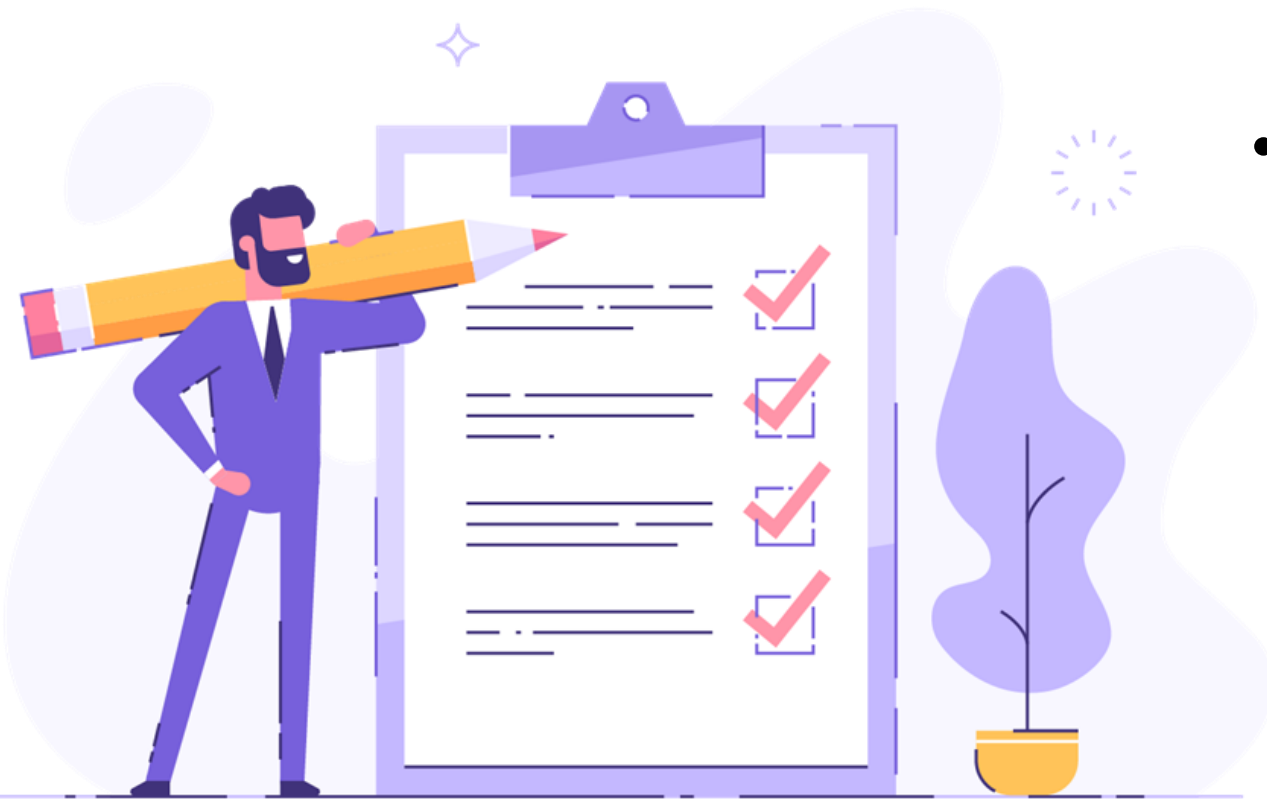


Disciplina: Engenharia e
Modelagem de Software
Professor: Felipe Lima



Objetivos de aprendizagem

- Modelagem unificada
- Diagramas de classe
- Relacionamento entre classes



Relacionamentos entre classes

- Em programação orientada a objetos, as classes interagem entre si para formar um sistema coeso e modular
- Essas interações são representadas por diferentes tipos de relacionamentos, que determinam como os objetos de uma classe usam ou dependem dos objetos de outra classe



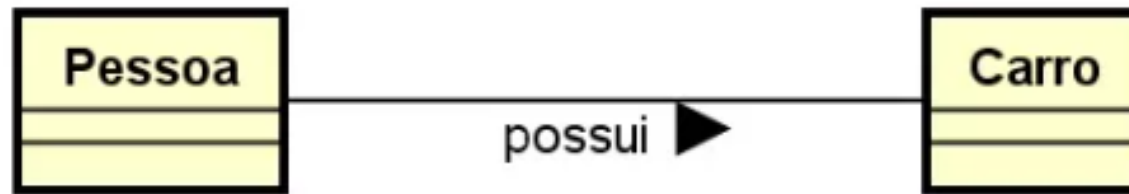
Associação

- Definição: Relacionamento onde uma classe usa outra
- A associação pode ser unidirecional ou bidirecional
- Exemplo: Um **Professor** leciona uma **Disciplina**
- Características:
 - As classes associadas podem existir de forma independente
 - Não implica dependência forte



Associação

- No diagrama, sua sintaxe é uma linha que liga as duas classes que representam os objetos relacionados



Tipos de Relacionamentos

- **Herança**: A classe filha herda comportamentos e características da classe pai
- **Associação**: Duas ou mais classes se relacionam, podendo colaborar sem dependências fortes
- **Agregação**: Relacionamento onde uma classe é parte de outra, mas pode existir de forma independente



Tipos de Relacionamentos

- **Composição**: Uma classe depende de outra para existir; se uma é destruída, a outra também é
- **Dependência**: Uma classe utiliza temporariamente outra para realizar uma ação específica



Tipos de Relacionamentos

- Facilita a organização e o reuso de código
- Melhora a manutenção e escalabilidade do sistema
- Ajuda a definir responsabilidades e a modularidade no design de software



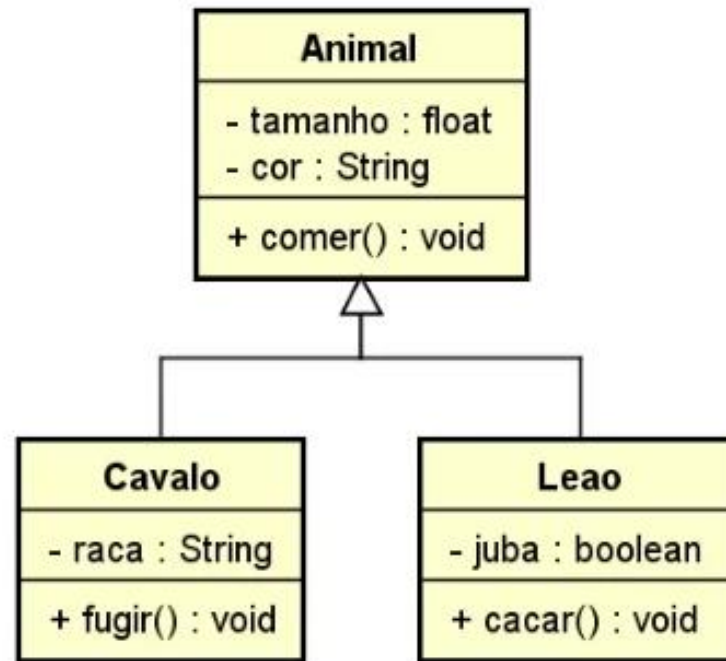
Herança

- Definição: Relação entre uma classe pai (superclasse) e uma classe filha (subclasse), onde a subclasse herda atributos e métodos
- Exemplo: **Cachorro** herda de **Animal**
- Características:
 - Reutilização de código
 - Facilita a manutenção e expansão do sistema



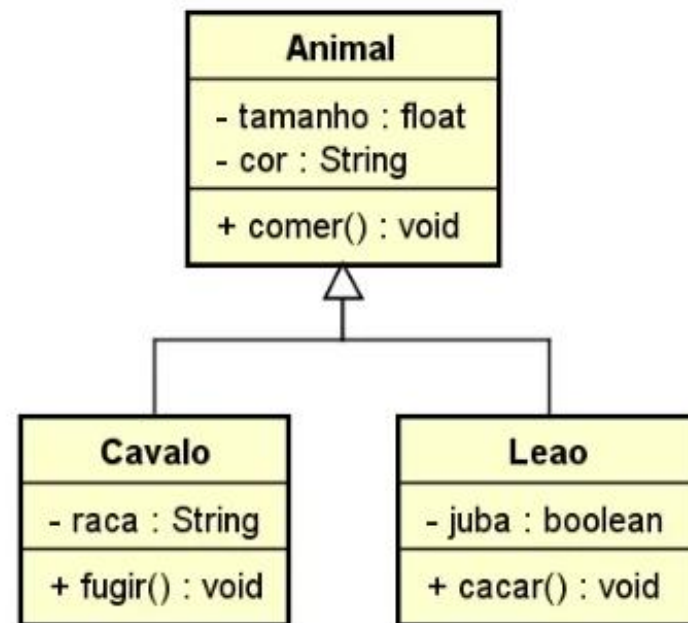
Herança

- Nesse exemplo houve a necessidade de criar uma classe mais geral, porque todos os animais possuem tamanho e cor e se alimentam de alguma forma



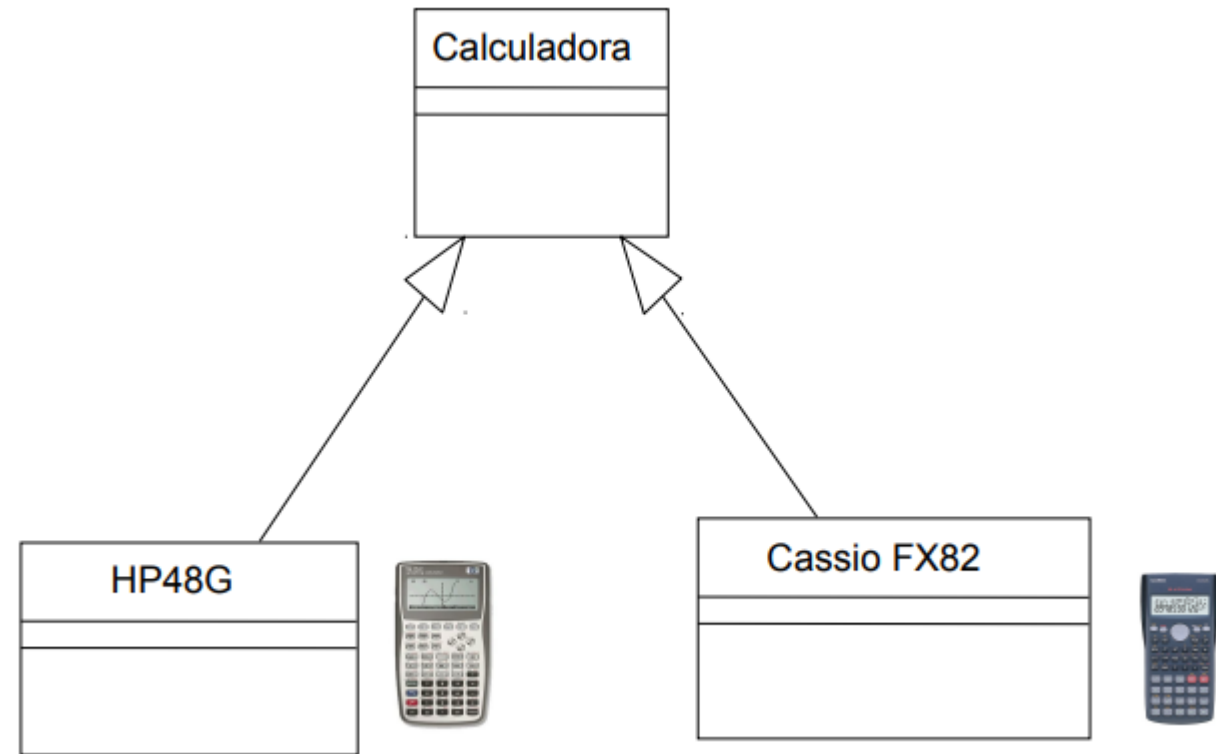
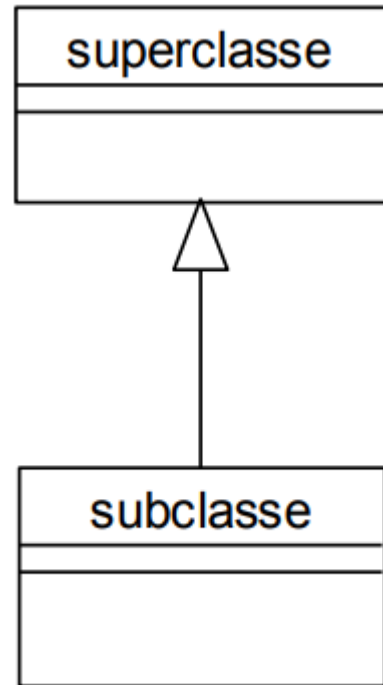
Herança

- Por outro lado, houve a necessidade de especializar alguns animais por possuírem características únicas



Herança

- Exemplos:



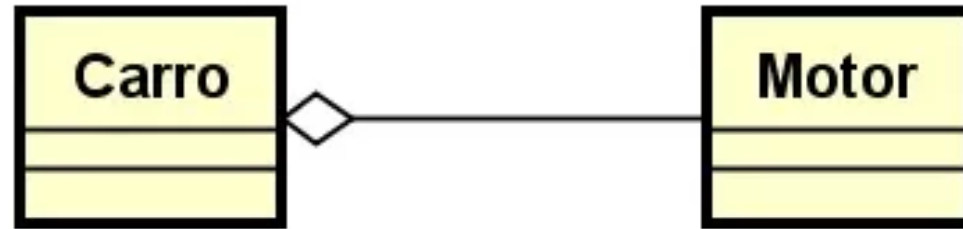
Agregação

- Definição: Relação todo-parte onde a parte pode existir independentemente do todo
- Exemplo: Um **Departamento** agrega **Professor**
- Características:
 - A parte (professor) não depende do todo (departamento) para existir



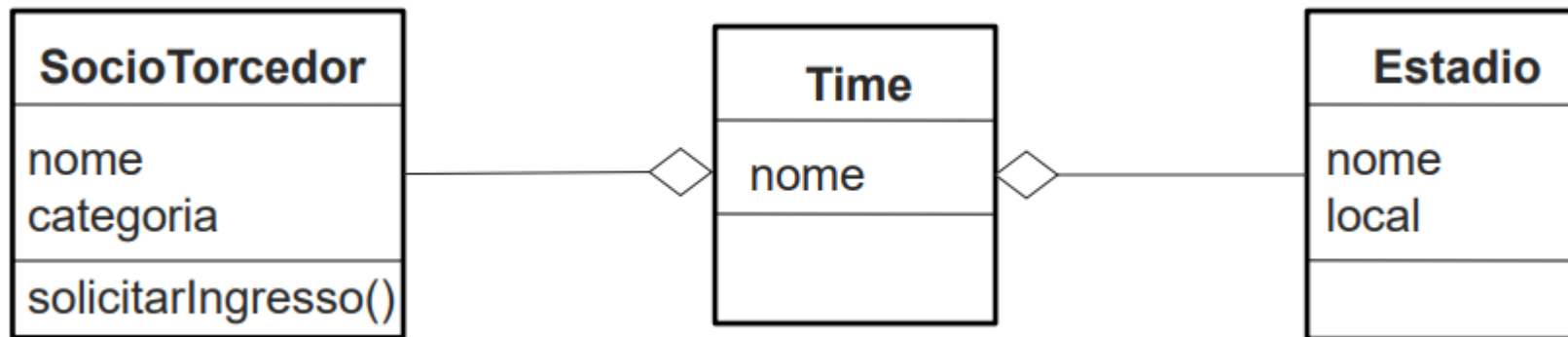
Agregação

- No diagrama, sua sintaxe é uma linha com um **losango branco** na classe que corresponde ao todo



Agregação

- No diagrama, sua sintaxe é uma linha com um **losango branco** na classe que corresponde ao todo



Composição

- Definição: Definição: Relação todo-parte, mas com dependência forte. Se o todo for destruído, a parte também é
- Exemplo: Uma Casa composta por Cômodos
- Características:
 - A parte (cômodos) depende do todo (casa) para existir



Composição

- No diagrama, sua sintaxe é a de uma linha com um **losango preto** na classe que corresponde ao todo



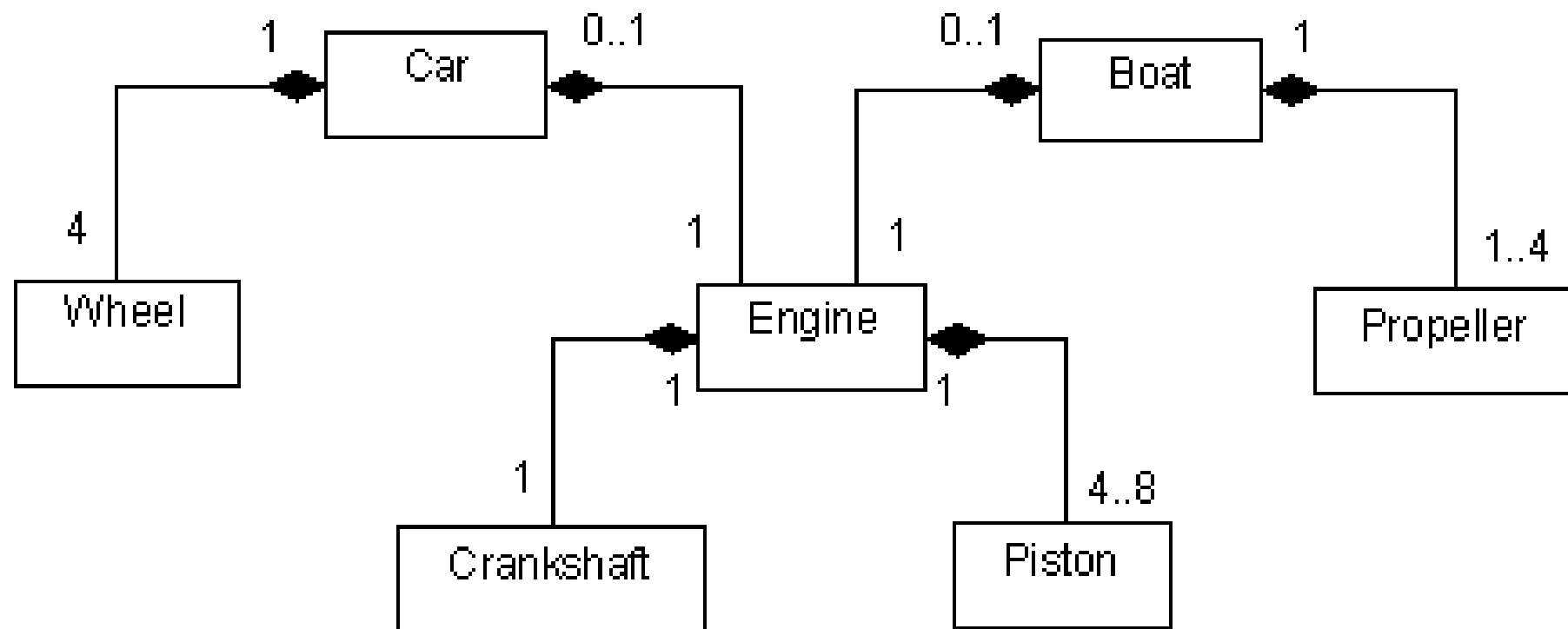
Composição

- No diagrama, sua sintaxe é a de uma linha com um **losango preto** na classe que corresponde ao todo



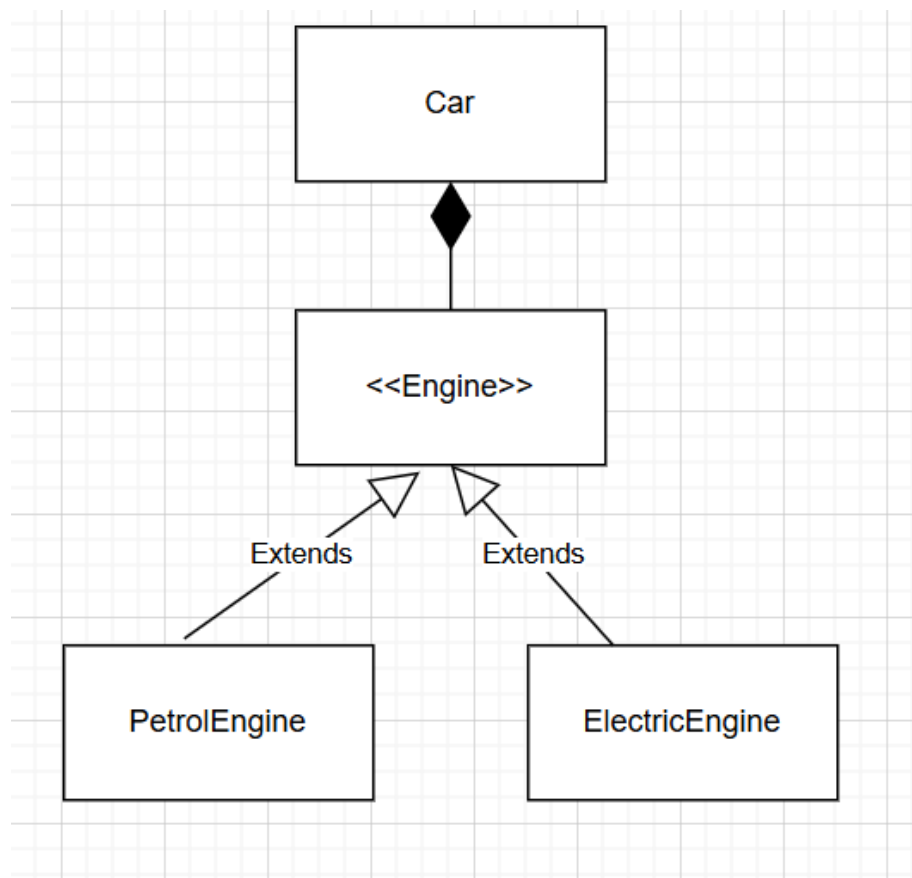
Composição

- No diagrama, sua sintaxe é a de uma linha com um **losango preto** na classe que corresponde ao todo



Composição

- No diagrama, sua sintaxe é a de uma linha com um **losango preto** na classe que corresponde ao todo



Composição

- Exemplo em Java

```
// Classe Motor
class Motor {
    private int potencia;

    public Motor(int potencia) {
        this.potencia = potencia;
    }

    public void ligar() {
        System.out.println("Motor ligado com potência de " + potencia + " cavalos.");
    }
}
```



Composição

- Exemplo em Java

```
// Classe Carro que usa a classe Motor
class Carro {
    private Motor motor; // Composição: um Carro TEM UM Motor

    public Carro(int potenciaMotor) {
        this.motor = new Motor(potenciaMotor);
    }

    public void ligarCarro() {
        System.out.println("Ligando o carro...");
        motor.ligar(); // O carro depende do motor para funcionar
    }
}
```



Composição

- Exemplo em Java

```
// Testando a composição
public class Main {
    public static void main(String[] args) {
        Carro meuCarro = new Carro(150);
        meuCarro.ligarCarro();
    }
}
```





WWW.UNIT.BR